

PROPOSTA EXECUTIVA • DEFESA ORAL

Arquitetura de **Big Data** para uma Frota Autônoma Conectada

Solução de dados resiliente, escalável e em tempo real que gera **valor mensurável** para o consórcio de locadoras — fundamentada na literatura de VLDB, Data Warehouse e Big Data.

GRUPO • COMPONENTES (NOME COMPLETO + DRE)

Izabela Lima da Silva	DRE 124156557
Caio Meirelles	DRE 122071557

GRAD.2026-1 • MAE016.15912 Prof. Milton Ramos Ramirez Rio de Janeiro • 2026.1

Estrutura da apresentação

01	Contexto e problema do negócio	COMUM	02	Valor para o cliente	COMUM
03	Objetivos e requisitos	COMUM	04	Metodologia do trabalho	COMUM
05	Fundamentos teóricos	COMUM	06	Do DW ao Big Data (+ extensão)	COMUM
07	Arquitetura de referência	COMUM	08	Camada de borda (Edge)	PARTE A
09	Ingestão — Kafka	PARTE A	10	Conectividade e ordenação	PARTE A
11	Lambda vs. Kappa	PARTE A	12	Lakehouse — Delta Lake	PARTE A
13	Processamento — Spark/Flink	PARTE A	14	Persistência poliglota (NoSQL)	PARTE B
15	Dashboard (Streamlit)	PARTE B	16	Cobrança e emergências	PARTE B
17	Concierge de IA por voz (PLN/RAG)	PARTE B	18	Casos de uso reais + rastreabilidade	COMUM
19	Custos, alternativas, referências e conclusões	COMUM			

Da locação manual à frota autônoma conectada

O **veículo autônomo** sai sozinho do pátio, atende o cliente e volta para higienização. Cada veículo é um **sensor sobre rodas**.

FONTE DE DADOS

Veículo instrumentado

Sensores + câmeras 360° + computador de bordo (*Edge*).

ESCALA

Frota simultânea

Milhares de veículos ao vivo, sob banda cara e finita.

AMBIÇÃO

Autonomia e IA

Reserva sem balcão, cobrança dinâmica, sinistros, concierge.

PROBLEMA CENTRAL

O DW sozinho não basta — é preciso **Big Data**: ingestão massiva, tempo real, persistência poliglota.

O que o consórcio ganha com a solução

UTILIZAÇÃO DA FROTA

Menos veículos ociosos

Markov + rotas antecipam a demanda e maximizam o giro.

↑ utilização

RECEITA

Cobrança justa e dinâmica

Acréscimos por km, consumo e infrações, com transparência.

↑ receita / locação

CUSTO OPERACIONAL

Sem balcão nem entrega

App + veículo autônomo cortam atendimento e logística.

↓ custo / locação

SINISTROS

Resposta mais rápida

Detecção na borda + dossiê automático aceleram o socorro.

↓ tempo de resposta

DISPONIBILIDADE

Manutenção preditiva

Antecipar panes reduz o veículo parado.

↓ downtime

EXPERIÊNCIA

Concierge por IA

Roteiro por voz diferencia e fideliza.

↑ satisfação / NPS

FIO CONDUTOR

Cada decisão técnica move um destes **indicadores** — a arquitetura é meio; o valor é o fim.

O que a solução precisa entregar

FUNCIONAIS

- **Dashboard:** frota, emergências, financeiro
- **Reserva** mais simples e eficiente
- **Cobrança automática** por uso, na devolução
- **Apoio a emergências** + dossiê de regulação
- *Extra:* **concierge por voz** (IA)

NÃO FUNCIONAIS • 5 DESAFIOS

- **Banda finita** — sem vídeo bruto contínuo
- **Alta concorrência** — sem perda de pacotes
- **Ordenação estrita** por veículo e timestamp
- **Resiliência** — falhas e reprocessamento
- **ACID** entre lote e streaming

Estes 5 desafios orientam todas as decisões a seguir.

Como a proposta foi construída

1

Requisitos

Do contexto e do enunciado.

2

Revisão bibliográfica

21 referências primárias, mapeadas à
ementa.

3

Decisão fundamentada

Trade-offs explícitos, com lastro teórico.

4

Seleção de componentes

Pelo padrão de acesso (poliglota).

5

Prova de conceito

docker-compose validado; DDL testado.

6

Continuidade com o DW

Estende a Aval. 02 — sem retrabalho.

Critério em cada camada: SLA · custo · resiliência · complexidade — com lastro citado.

Ombros de gigantes: as bases do projeto

Cada camada tem lastro na literatura — pelas 4 partes da ementa.

PARTE	PRINCÍPIO	REFERÊNCIA	ONDE APLICAMOS
I · VLDB	Falha é a norma; localidade	Ghemawat 2003; Dean 2004	Réplicas, Edge, Spark
I · Tempo	Ordem / consistência externa	Corbett 2012	Ordenação por veículo+ts
II · DW / Lakehouse	OLAP ≠ OLTP; CUBE; Lakehouse	Chaudhuri 1997; Gray 1997; Armbrust 2021	DW vira camada Gold
III · Streaming	Log particionado; watermarks	Kreps 2011; Carbone 2015; Zaharia 2013	Kafka, Flink, Spark
III · IA	Geração ancorada (RAG)	Lewis 2020; Guo 2022	Concierge e dossiês
IV · NoSQL / CAP	CAP; BASE vs ACID; LSM	Gilbert 2002; DeCandia 2007; Lakshman 2009	Persistência poliglota

Do Data Warehouse ao Big Data – e a extensão do modelo

AVALIAÇÃO 02 • OLTP → OLAP [CHAUDHURI & DAYAL 1997]

Esquema estrela (PostgreSQL)

- Fatos: **Reserva, Locação, Movimentação** (fact constellation)
- 5 dimensões conformadas; Pátio *role-playing*
- 4 fontes OLTP; previsão de pátios via **Markov**

AVALIAÇÃO 03 • O DW VIRA A CAMADA GOLD

Extensão dimensional

- Novos fatos: **Telemetria, Cobrança, Sinistro, Manutenção**
- Novas dims: Tempo_Detalhe, Sensor (SCD2), TipoEvento, FaixaCondução
- Carregados por **ELT do Silver** (Flink/Spark)

ESTRATÉGIA – LAKEHOUSE [ARMBRUST 2021]

Não substituir, e sim **envolver**: o esquema estrela vira a camada **Gold** do Lakehouse, alimentada pelo streaming.

Pipeline fim a fim: ingestão → processamento → armazenamento → visualização

1 · BORDA

Edge Computing

Pré-processa e agrega; 2 fluxos.



2 · INGESTÃO

Kafka

Tópicos isolados; ordem por veículo.



3 · PROCESSAMENTO

Spark · Flink

Emergências e análise contínua.

4 · LAKEHOUSE

Delta Lake

Bronze/Silver/Gold · ACID.



5 · PERSISTÊNCIA

NoSQL poliglota

Cassandra · MongoDB · Redis.



6 · SERVING

Dashboard · IA

Streamlit · cobrança · concierge.

HOT PATH

Colisão → alerta imediato → emergência.

WARM PATH

Telemetria 1s → análise e roteirização.

COLD PATH

Histórico → relatórios e manutenção.

Filtrar na borda: só o relevante trafega pela rede

Fundamento: **localidade** — a pré-agregação no veículo é o *combiner* do MapReduce na borda, poupando banda. [Dean & Ghemawat 2004]

FLUXO DE ROTINA • WARM PATH

Telemetria agregada

- Velocidade, GPS, bateria, combustível, sensores
- Agregada e enviada a cada **1 s (ou mais)**
- Binário compacto: Avro sobre MQTT

FLUXO CRÍTICO • HOT PATH

Alerta imediato de sinistro

- Detecção de colisão **na borda** (reação sub-segundo)
- Prioriza a rede e envia *payload* de emergência
- *Ring buffer*: ~30 s de dados brutos + metadados

VALOR

Corta o **maior custo** (banda) e a resposta não espera a nuvem. O vídeo bruto fica no veículo.

Apache Kafka como log distribuído particionado

Log particionado, consumo *pull* por offset, retenção — buffer durável com **ordem estrita na partição**.

[Kreps 2011]

TÓPICOS ISOLADOS

Separar telemetria de alarmes

telemetry.raw

alerts.critical

trips.events

billing.events

O alarme crítico **nunca fica preso** atrás da fila de telemetria.

GARANTIAS

Ordem e resiliência

- Partição por `vehicle_id` → ordem sem coordenação global
- Replicação + ISR → tolerância a falhas [Ghemawat 2003]
- At-least-once + dedup por chave
- Avro + Schema Registry → contrato de esquema

Rede oscilante e mensagens fora de ordem

Túneis e zonas de sombra entregam dados fora de ordem. Três camadas resolvem — o que o Spanner faz com TrueTime, o Flink faz com *watermarks*. [Corbett 2012; Carbone 2015]

BORDA

Compressão + buffer

Avro comprime; *store-and-forward* reenvia sem perda.

KAFKA

Ordenação por chave

Partição por `vehicle_id` mantém a ordem por veículo.

FLINK

Event-time + watermarks

Processa por *event-time*; *watermarks* reordenam atrasados. [Fragkoulis 2020]

RESULTADO

Ordenação estrita por veículo e timestamp — mesmo sob estresse e rede intermitente.

Lambda vs. Kappa – e a decisão

CRITÉRIO	LAMBDA	KAPPA
Ideia	Batch (exato) + velocidade + serving	Tudo é stream; replay do log
Base de código	Dupla (dois sistemas)	Única
Consistência	Risco de divergência	Uma verdade
Manutenção	Alta	Menor
Reprocessar histórico	Camada batch nativa	Replay + Lakehouse

DECISÃO

O stream durável do Kafka unifica os caminhos e **elimina a dupla implementação** — adotamos **Kappa**, cobrindo o lote pesado com o Lakehouse. [Fragkoulis 2020; Carbone 2015]

Kappa realizado por Lakehouse com Delta Lake

O object store é *key-value* sem atomicidade cross-key; o **log de transações do Delta** traz ACID serializável, *data skipping* e *time travel* ao data lake. [Armbrust 2020, 2021]

BRONZE

Bruto

Ingerido do Kafka; append-only, auditável.

>

SILVER

Limpo e conformado

Deduplicado, validado, tipado.

>

GOLD

Marts de negócio

O DW estrela estendido (BI).

ACID

Batch e streaming leem a mesma verdade.

TIME TRAVEL

Auditar versões e montar dossiês reproduzíveis.

MOTOR

Spark SQL + Catalyst [Armbrust 2015].

Três regimes para três SLAs distintos

BATCH / COLD · HORAS

Apache Spark

RDD, tolerância por **linhagem**; memória 20×. [Zaharia 2012]

- Consolida do Delta Bronze
- Manutenção preditiva; Markov
- Generaliza o **MapReduce** [Dean 2004]

STREAM · SEGUNDOS

Spark Streaming

Micro-batches com **exactly-once** idempotente. [Zaharia 2013]

- Hot path das emergências
- Orquestra a resposta

STREAM · MILISSEGUNDOS

Apache Flink

Dataflow único; *ABS* sem parar a execução. [Carbone 2015]

- Janelas + event-time + watermarks
- Rotas, congestão, score

POR QUE SEPARAR

Vazão histórica ≠ emergência ≠ latência mínima. A reação sub-segundo é na borda; a nuvem orquestra.

Cada dado no banco certo para seu padrão de acesso

Raiz: o **teorema CAP** — sob partição, escolhe-se consistência ou disponibilidade. Cada subserviço, seu trade-off. [Gilbert & Lynch 2002; Cattell 2011]

WIDE-COLUMN · AP

Cassandra

Telemetria. Escrita massiva (LSM); *clustering* por ts.
[Lakshman 2009]

DOCUMENTO

MongoDB

Cadastrais e dossiês.
Esquema flexível.

KEY-VALUE · RAM

Redis

Cache quente. Estado ao vivo, rotas (sub-ms).

RELACIONAL · CP

PostgreSQL / Delta

Fonte da verdade. Cobrança (ACID), DW/Gold.

MAPEAMENTO CAP

Telemetria = **AP**; cobrança/dossiê = **CP** — o mesmo trade-off do carrinho vs. checkout do Dynamo.

Painel de monitoramento em Streamlit

OLAP sobre a **Gold** (nunca OLTP), ao vivo via **Redis**. Cubo frota × tempo × pátio × empresa.
[Chaudhuri 1997; Gray 1997]

FROTA

Mapa ao vivo, veículos ativos, bateria média, alertas mecânicos.

RESERVAS & LOCAÇÕES

Funil de reservas e locações em andamento.

FINANCEIRO

Cobranças, receita e ajustes por locação.

EMERGÊNCIAS

Triagem que pisca em vermelho no payload crítico.

VIAGENS

Trajetos em curso, ETA, congestionamentos.

OFICINA

Manutenção preditiva e fila de higienização.

AO VIVO

Quase tempo real via Redis + auto-refresh.

HISTÓRICO

KPIs e tendências das marts Gold.

Cobrança automática e apoio a sinistros

COBRANÇA DINÂMICA · VALOR AO CLIENTE

Acionada na devolução

Valor = base ± km, tempo, consumo, infrações, score.

Exactly-once evita cobrar em dobro; auditável por time travel. [Zaharia 2013]

↑ receita · ↓ contestações

EMERGÊNCIAS · HOT PATH

De colisão a dossiê de regulação

borda → alerts.critical → Spark → resposta

— Guincho, seguradora, substituto

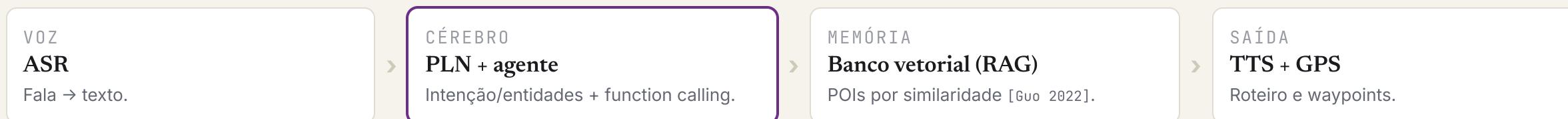
— Dossiê citável (sensores + timeline)

↓ tempo de resposta e de regulação

Ponte: a detecção (Parte A) alimenta a resposta e o dashboard (Parte B).

Concierge por voz: PLN + RAG (sem LLM paga)

O **RAG** ancora a geração em conhecimento recuperável: reduz alucinação e dá **proveniência** — a mesma base dos dossiês. [Lewis 2020]



EXEMPLO — "APÓS A REUNIÃO..."

Almoço típico + estacionamento fácil + arte nos gostos do cliente → o agente consulta localização, POIs e agenda, monta o roteiro e **injeta os waypoints no GPS**. Privacidade: consentimento, PII, *wake-word*.

Três cenários: onde a modelagem cria valor

CRÍTICO · RESPOSTA RÁPIDA

Colisão em túnel

Borda detecta (sub-segundo) → Spark orquestra guincho, seguradora, substituto; Flink reordena a telemetria retida no túnel.

Acende: **Fato_Sinistro**, Dim_Sensor (SCD-2), Dim_TipoEvento, **Dim_Tempo_Detalhe**.

↓ tempo de resposta

USUAL · FINANCEIRO

Devolução com condução agressiva

Flink computa o score →

Dim_FaixaConducao "Agressivo"; a cobrança soma km, consumo e a infração.

Acende: **Fato_Cobranca**, Fato_Telemetria_Diaria, Dim_FaixaConducao, Dim_TipoEvento.

↑ receita / locação

ANALÍTICO · IA

Pico no aeroporto + pane + concierge

Markov prevê a migração ao Galeão → reposiciona vazios; o batch prevê falha → agenda; o concierge monta o roteiro.

Acende: Fato_Movimentacao, **Fato_Manutencao**, Fato_Telemetria_Diaria, Dim_Patio, Dim_Tempo_Detalhe.

↑ utilização · ↓ downtime · ↑ NPS

FUNDAMENTAÇÃO

Três casos **justificam a extensão** (Dim_Tempo_Detalhe, Fato_Cobranca, Fato_Manutencao). A modelagem aparece no crítico, no usual e no analítico.

Onde cada entidade modelada aparece

ENTIDADE + = EXTENSÃO	1 · COLISÃO CRÍTICO	2 · COBRANÇA USUAL	3 · FROTA ANALÍTICO
Fato_Sinistro +	dossiê + resposta	—	—
Fato_Cobranca +	recálculo da locação	valor ± acréscimos	—
Fato_Telemetria_Diaria +	buffer pré-impacto	km, velocidade, eventos	base do score / manutenção
Fato_Manutencao +	—	—	prob. de falha → agenda
Fato_Movimentacao	despacho ao pátio	—	Markov → reposicionamento
Fato_Reserva / Locação	veículo substituto	locação faturada	pool de disponíveis
Dim_Sensor + SCD-2	firmware na hora do sinistro	—	saúde do sensor
Dim_TipoEvento +	colisão	violação de trânsito	pane / superaquecimento
Dim_FaixaConducao +	—	Agressivo → tarifa	perfil por veículo
Dim_Tempo_Detalhe +	minuto 18:23	janelas da viagem	faixa horária / pico
Dim_Patio · Veículo · Cliente	pátio próximo · quem	retirada / devolução	origem / destino

Custo sob controle por design

BANDA / EGRESS

Filtrar na borda

Agregar e não enviar vídeo bruto corta o maior custo recorrente da frota móvel.

COMPUTE

Elasticidade

Autoscaling + instâncias spot/preemptíveis; clusters efêmeros no batch.

STORAGE

Tiering

Quente (Redis/Cassandra) vs. frio (Delta/object store), com TTL e Bronze→Gold.

PROVA DE CONCEITO EXECUTÁVEL

Sistema **docker-compose** (Kafka, Flink, Spark, Cassandra, Mongo, Redis, MinIO, Streamlit) validado; DDL do DW estendido testado em PostgreSQL. Free tiers: Dataproc/Flink Playground, Databricks, Atlas, Astra, Redis, DynamoDB.

Alternativas consideradas e descartadas

OPÇÃO DESCARTADA	POR QUE NÃO, NESTE CONTEXTO
Somente Data Warehouse	Batch e schema-on-write não dão tempo real nem absorvem sensores semiestruturados de alto volume.
Somente processamento batch	Emergências e roteirização exigem latência de segundos/milissegundos.
Arquitetura Lambda	Base de código dupla e risco de divergência; o problema é streaming-first.
Um único banco relacional	Não escala a escrita massiva de telemetria nem entrega cache sub-ms (CAP).
Stream de vídeo bruto pela rede	Banda móvel cara e finita; o vídeo fica na borda e sobe só o essencial.
Ingestão HTTP síncrona no banco	Sem contrapressão nem replay; a frota simultânea causaria perda de pacotes.

Fundamentos – referências primárias

Armbrust, M. et al. (2015). Spark SQL. SIGMOD.

Armbrust, M. et al. (2020). Delta Lake. PVLDB.

Armbrust, M. et al. (2021). Lakehouse. CIDR.

Carbone, P. et al. (2015). Apache Flink. IEEE Data Eng. Bull. + Lightweight Asynchronous Snapshots. arXiv:1506.08603.

Cattell, R. (2011). Scalable SQL and NoSQL Data Stores. SIGMOD Record.

Chang, F. et al. (2006). Bigtable. OSDI.

Chaudhuri, S. & Dayal, U. (1997). Overview of Data Warehousing and OLAP. SIGMOD Record.

Corbett, J. C. et al. (2012). Spanner. OSDI.

Dean, J. & Ghemawat, S. (2004). MapReduce. OSDI.

DeCandia, G. et al. (2007). Dynamo. SOSP.

Fragkoulis, M. et al. (2020). Survey on Stream Processing. arXiv:2008.00842.

Ghemawat, S. et al. (2003). The Google File System. SOSP.

Gilbert, S. & Lynch, N. (2002). Brewer's Conjecture (CAP). ACM SIGACT.

Gray, J. et al. (1997). Data Cube. DMKD.

Guo, R. et al. (2022). Manu: Cloud Native Vector DB. PVLDB.

Kreps, J. et al. (2011). Kafka. NetDB.

Lakshman, A. & Malik, P. (2009). Cassandra. LADIS/SIGOPS.

Lewis, P. et al. (2020). Retrieval-Augmented Generation. NeurIPS.

Zaharia, M. et al. (2012). Resilient Distributed Datasets (RDD). NSDI.

Zaharia, M. et al. (2013). Discretized Streams. SOSP.

Síntese: fundamentos → arquitetura → valor

- **Fundamentada** — cada camada ancorada em papers seminais (VLDB, DW, streaming, NoSQL).
- **Borda + Kafka + Kappa/Delta** — banda poupada, ordem por veículo, ACID lote↔stream.
- **Três regimes** — Spark (batch/emergências) e Flink (rotas).
- **Poliglota** — CAP aplicado: Cassandra, MongoDB, Redis, PostgreSQL/Delta.
- **DW estendido** — vira a camada Gold; novos fatos de telemetria/cobrança/sinistro.
- **Serving + IA** — dashboard, cobrança auditável, dossiês e concierge RAG.
- **Valor ao cliente** — ↑ utilização e receita, ↓ custo, downtime e tempo de resposta.
- **Executável** — sistema em docker-compose, validado.

CONCLUSÃO

Uma arquitetura **resiliente, escalável e em tempo real**, teoricamente fundamentada, que envolve o DW da Avaliação 02 e entrega **valor mensurável** ao consórcio com custo de nuvem sob controle.

Obrigado. Perguntas? Izabela Lima da Silva · Caio Meirelles · 2026.1